



Republic of the Philippines
UNIVERSITY OF SCIENCE AND TECHNOLOGY OF SOUTHERN PHILIPPINES
Department of Computer Science
Cagayan de Oro Campus



A Bivariate Reliability Degradation Index: Algorithmic State-Transition Evaluation

Amor Bajarla, Franco Galendez, Arjay Libutan, Jude Rabaya, Nicole Tuazon

Department of Computer Science, College of Information Technology and Computing
University of Science and Technology of Southern Philippines

1. INTRODUCTION OF THE STUDY

Intelligent systems deployed in resource-constrained edge computing environments operate under continuous environmental variability. Over the past decade, the architectural paradigm of internet of things deployments has shifted massively from centralized cloud processing to decentralized edge computing. This shift is driven by the need to minimize network latency, reduce bandwidth consumption, and ensure data privacy. However, deploying complex artificial intelligence models directly onto local nodes introduces severe physical constraints. As artificial intelligence models scale in complexity, deploying them in edge environments exposes emerging multidisciplinary challenges regarding system stability and resource allocation (Dwivedi et al., 2021). These edge devices typically lack the specialized tensor processing units or discrete graphics accelerators found in data centers, forcing standard central processing units to handle heavy mathematical workloads.

When input data degrades due to hardware wear, environmental noise, or unexpected distribution shifts, these systems typically suffer a dual penalty. First, feature extraction algorithms struggle to resolve corrupted data, leading to a sharp decline in predictive accuracy. Second, and critically, this struggle triggers a massive spike in computational latency (Khoei & Kaabouch, 2023). A system forced to over-process noisy data consumes excessive CPU cycles, leading to thermal throttling and memory bottlenecks. Traditionally, machine learning evaluation has prioritized static peak accuracy achieved under perfectly controlled laboratory conditions, ignoring the practical algorithms required for real-world applications (Sarker, 2021). This singular focus on predictive power creates a false sense of security, as models that perform flawlessly on clean data often fail catastrophically when introduced to real-world edge friction.

This paper shifts the focus toward sustained operational stability by proposing a Bivariate Reliability Degradation Index. Grounded in algorithmic state-machine logic rather than continuous calculus, this index evaluates model resilience based on a Time-to-Failure metric. By tracking how a system degrades through specific operational states when exposed to synthetic distribution shifts, this study provides a highly practical framework for quantifying reliability under hard physical hardware constraints. The ultimate goal is to bridge the gap between theoretical model performance and actual hardware survival, providing software engineers with a discrete tool to measure exact operational limits before deployment.

2. PROBLEM STATEMENT

Current evaluation methodologies are fundamentally insufficient for capturing the dual nature of performance degradation in real-world edge deployments. Standard metrics such as the F1 score or top-1 accuracy provide static benchmarks under fixed laboratory conditions. These metrics treat model inference as an isolated mathematical event, completely detaching the algorithm from the physical hardware executing it. They fail to account for how computational latency scales exponentially as input accuracy drops, a critical oversight in real-world engineering applications (Kumar & Rani, 2024). When an edge device encounters noisy data, the neural network requires more time to process anomalous features, thereby stealing processing power from other vital system operations.

While deep learning has revolutionized computer vision tasks (Voulodimos et al., 2022), building robust deep learning models that maintain efficiency under environmental noise remains a significant and unsolved challenge (Liu & Jin, 2023). The reliance on static datasets obscures the reality that software failures on edge nodes are rarely graceful. Instead of merely outputting an incorrect classification, overburdened models cause the central processing unit to hang, requiring hard system reboots. This is especially problematic in autonomous systems where real-time decision making is critical.

Consequently, there exists a critical lack of a standardized, algorithmic metric for quantifying bivariate performance degradation encompassing both accuracy and latency under strict, CPU-bound hardware constraints. Existing reliability models rely heavily on complex, multi-objective optimization calculus, making them difficult to implement programmatically in standard edge software pipelines. This gap complicates efforts to deploy resilient, fail-safe edge-computing systems that must autonomously prevent complete computational lockups when processing heavily degraded environmental inputs.

3. OBJECTIVES AND SIGNIFICANCE OF THE STUDY

Objectives

The primary aim of this study is to establish an algorithmic evaluation framework for edge-deployed machine learning models. By moving away from theoretical calculus

and embracing discrete logic, this research seeks to create a highly deployable metric. Specifically, the study seeks to:

- Formulate a discrete Time-to-Failure metric that quantifies system degradation by measuring the exact number of frames processed before predefined accuracy and latency thresholds are breached, providing a modern monitoring tool for edge processes (Gopaluni et al., 2022).
- Implement a 5-state Non-deterministic Finite Automaton to dynamically categorize operational health, from optimal performance to terminal system failure, allowing for state reversibility as input quality fluctuates.
- Benchmark lightweight architectures under strict hardware constraints by comparing the degradation resilience of MobileNetV3 and ShuffleNet against a standard ResNet-18 baseline, executing all inferences entirely on CPU-bound hardware, reflecting the strict energy limits of modern microgrid systems (Joshi et al., 2023).
- Validate a distributed edge telemetry pipeline, demonstrating that system logging and state tracking can be offloaded to a remote database via secure networking without corrupting local inference latency metrics.

Significance of the Study

The findings of this study directly benefit system architects, DevOps engineers, and software developers building decentralized internet of things deployments. By replacing theoretical optimization math with actionable, discrete code blocks, the research delivers a deployable circuit breaker for failing software. In the contemporary computing landscape, theoretical accuracy is secondary to system uptime. Creating a state machine that tracks exact performance degradation allows developers to trigger fail-safes before a total computational crash occurs.

This methodology is particularly valuable for technology firms in Northern Mindanao and Cagayan de Oro operating constrained local server nodes. Local infrastructure often relies on cost-effective, standard commercial hardware rather than enterprise-grade tensor processors. Providing these engineers with a concrete tool to maximize resource management ensures that regional tech deployments can maintain high reliability without requiring massive hardware budgets. The study bridges the gap between academic evaluation and industry necessity, offering a robust engineering standard for edge deployment.

4. COMPREHENSIVE LITERATURE REVIEW

Hardware Limitations in Edge Deployment

The transition of computational tasks from centralized cloud servers to decentralized edge nodes represents a major shift in modern network architecture. Edge computing promises significantly reduced network latency and improved data privacy by processing information locally at the source of data generation. However, this decentralized approach inherently forces complex algorithms to execute on hardware that was not explicitly designed for high-performance computing. A comprehensive survey of machine learning accelerators reveals that standard local nodes often lack the specialized tensor processing units required to efficiently run modern neural networks, creating a severe bottleneck for real-world artificial intelligence deployments (Reuther et al., 2021).

Recent literature heavily emphasizes that deploying machine learning on these edge nodes exposes systems to critical physical constraints. Jouini et al. (2024) conducted a comprehensive survey on machine learning in edge computing, specifically noting how intensive artificial intelligence tasks consistently overwhelm local processing power. Their research outlines the myriad issues surrounding power consumption, memory allocation, and processor throttling that occur when heavy models are deployed outside of controlled laboratory environments. When a model requires more computational resources than the physical hardware can provide, the system does not simply return a less accurate prediction; it experiences severe latency spikes that cascade into an application hang.

Building upon the identification of these bottlenecks, Khouas et al. (2024) explored the specific challenges of training and executing machine learning models directly at the edge. Their findings validate the necessity of evaluating systems within strict physical boundaries, rather than relying on theoretical benchmarks. Algorithms that perform exceptionally well in cloud environments often fail unpredictably on edge devices due to the lack of dynamic resource allocation. This gap between theoretical capability and practical deployment proves that evaluating a model based solely on its predictive power is fundamentally flawed if the hardware cannot sustain the required mathematical operations.

These studies directly validate the core methodology of this research. By explicitly isolating all model inference to a CPU-bound architecture, this study mirrors the exact hardware limitations documented by current edge deployment models. Ignoring the latency metric in such environments inevitably leads to total system failure. Therefore, developing a metric that penalizes computational inefficiency is not just a theoretical improvement, but an operational necessity for local technology infrastructure.

Rethinking Generalization and Robustness

Evaluating a machine learning model exclusively on clean, undisturbed data does not reflect operational reality. Historically, neural network evaluation relied heavily on standard validation sets that perfectly mirrored the training data distribution. However, Zhang et al. (2021) demonstrated that understanding deep learning requires a fundamental rethinking of generalization, proving that large capacity models often memorize training sets rather than learning underlying, robust features. When these models are deployed into physical environments, they immediately encounter out-of-distribution data, creating a false sense of security where lab-tested models fail catastrophically when encountering basic environmental noise.

To combat this, the academic focus has shifted heavily toward domain generalization. Zhou et al. (2022) highlighted in their comprehensive survey that computer vision models struggle significantly when domain shifts occur, such as changes in lighting, weather, or sensor degradation. This issue is amplified in decentralized edge and internet of things networks. A model cannot simply be accurate; it must maintain that accuracy across a wide spectrum of visual variations and occlusions without requiring immediate retraining.

This paradigm shift validates the necessity of utilizing synthetic distribution shifts to stress-test system boundaries. As established by Hendrycks and Dietterich (2019), evaluating deep neural network robustness requires standardized benchmarks like CIFAR-10-C, which explicitly simulate these exact real-world domain shifts by applying algorithmic corruptions, such as Gaussian noise, motion blur, and contrast changes, at escalating severity levels. Measuring how a model generalizes against predefined noise severities directly informs its resilience in unpredictable physical environments. By systematically injecting these perturbations, developers can map the exact threshold where a model stops generalizing and begins to output random predictions.

Therefore, evaluating a system using the proposed Time-to-Failure metric across the CIFAR-10-C dataset provides a rigorous assessment compared to static accuracy. It forces the system to prove its generalization capabilities under stress. By tracking the exact state transitions from optimal operation to terminal failure, the Bivariate RDI objectively measures how well a model handles representation drift, transforming abstract robustness into a measurable, algorithmic engineering standard.

Data-Efficient Architectures

When computational processing power is strictly limited, the choice of neural network architecture becomes the primary determinant of system longevity. As

internet of things deployments scale globally, the demand for local inference has risen dramatically to improve latency, bandwidth efficiency, and data privacy. Federated learning techniques have shown that optimizing local architectures can drastically reduce the communication and computational overhead required for remote networks (Kairouz et al., 2021). Attempting to execute massive architectures on standard edge nodes results in severe throttling, rapid battery depletion, and unacceptable inference delays.

Addressing these physical constraints requires a deliberate pivot toward streamlined, data-efficient architectures. Research on efficient convolutional neural networks emphasizes that practical edge deployment should consider not only the number of parameters but also memory access, parallelism, and actual execution speed on the target hardware (Ma et al., 2018). Touvron et al. (2021) highlighted the absolute necessity of training data-efficient models that maintain high operational capacity without demanding excessive computational overhead. Over-parametric models are increasingly recognized as prone to overfitting and instability under constrained conditions, making lightweight architectures the preferred standard for affordable inference.

The integration of these lightweight models necessitates streamlined processing pipelines. Research by Carion et al. (2021) into end-to-end architectures strongly supports the elimination of complex, multi-stage processing that traditionally bogs down execution. Modern edge deployments are increasingly utilizing modular designs and lightweight neural processing units designed specifically for small-model vision and anomaly detection. By reducing the mathematical operations required per frame, these architectures allow the local processor to execute inference cycles rapidly without triggering thermal throttling or computational hangs.

This established body of work directly reinforces the decision to benchmark inherently lightweight networks within this study. Evaluating architectures such as MobileNetV3 and ShuffleNet against standard heavy baselines, like ResNet-18, provides empirical evidence of how architectural efficiency dictates system survival. Under the proposed Time-to-Failure metric, data-efficient models are hypothesized to sustain operations significantly longer in constrained environments. By proving this variance empirically, the study provides a concrete framework for local technology infrastructure teams to select the appropriate model architecture before deployment.

Network Logging and Edge Telemetry

Evaluating latency accurately on a constrained edge node requires the strict isolation of central processing unit resources. When a system is tasked with executing

intense machine learning inference, any background operations can severely corrupt the performance metrics. Zhang, Ding, and Wang (2023) highlighted the distinct separation of responsibilities across cloud, edge, and internet of things computing paradigms, advocating for distributed architectures to manage system loads. If an edge device attempts to run predictive inference while simultaneously writing extensive state-tracking logs to its local storage, the resulting input and output overhead will artificially inflate the inference latency measurements.

To solve this bottleneck, modern deployments rely heavily on distributed edge telemetry. The push toward edge-native architectures has made remote telemetry and distributed logging standard practices for maintaining fleet health. Modern platforms emphasize the capability to capture telemetry remotely and manage deployments through centralized cloud governance without burdening the localized edge hardware. This allows the local node to dedicate its full computational bandwidth to executing the machine learning model, transferring the administrative burden to a remote database via secure networking protocols.

This study's methodology directly adopts these distributed telemetry principles to validate the Bivariate Reliability Degradation Index. By separating the local inference node from the remote state-tracking database, the research guarantees that the recorded latency spikes are exclusively the result of algorithmic degradation and feature extraction struggles. This strict isolation validates the Time-to-Failure metric as an accurate reflection of the hardware's limitations. Furthermore, proving that this distributed logging setup operates successfully provides a blueprint for technology firms to monitor decentralized networks efficiently.

Beyond monitoring telemetry and handling hardware boundaries, maintaining operational integrity in autonomous edge systems requires formal runtime verification and discrete state management. Traditional software monitoring often relies on reactive error handling, which triggers only after a catastrophic system crash has already occurred. In contrast, finite state machines and automaton frameworks allow systems to track health transitions proactively before terminal failure happens. Recent studies in smart manufacturing and embedded edge reliability emphasize that utilizing formalized state validation models provides a systematic approach to categorize runtime performance states and manage data streams or process timers under fluctuating workloads (Afsar et al., 2023). By structuring operational degradation into discrete state transitions, an edge device can dynamically recognize performance drift and initiate preventive actions before CPU-bound processing pipelines suffer total lockups. This state-machine approach directly supports the foundational architecture

of the Bivariate Reliability Degradation Index, transforming abstract performance decline into predictable, manageable operational bounds.

5. METHODOLOGY

This section presents the algorithmic and experimental framework used to operationalize the Bivariate Reliability Degradation Index. The methodology is organized into six subsections: research design, the formal state model, the Time-to-Failure formulation, the index computation, the experimental pipeline, and the distributed telemetry architecture. Each component is designed to directly serve the paper's central intention, replacing static accuracy benchmarks with a discrete, algorithmic measure of sustained operational stability under strict CPU-bound edge constraints.

5.1 Research Design

The study adopts a quantitative, experimental design grounded in algorithmic state-machine logic rather than continuous calculus. The unit of analysis is the individual inference frame. Each frame represents a discrete observation of the system's bivariate state, defined by the pair of predictive accuracy and inference latency at a given time. The design follows a rigorous three-phase protocol to isolate variables and measure true performance. First, Baseline Profiling is conducted where each architecture is evaluated on clean CIFAR-10 data to establish nominal accuracy and latency references. Second, Controlled Degradation is applied where inputs are progressively corrupted using CIFAR-10-C corruption types at escalating severity levels, accurately simulating real-world domain shifts. Finally, State-Transition Tracking is executed where the Non-deterministic Finite Automaton monitors the bivariate state in real time, logging every state transition until the system reaches Terminal Failure or the corruption sequence concludes.

5.2 Formal NFA State Model

The operational health of the system is tracked using a 5-state Non-deterministic Finite Automaton. Utilizing this structure instead of a standard deterministic model allows the system to possess state reversibility, meaning the program can recover to a healthier tier of operation if input data quality temporarily

improves. The automaton evaluates current frame data against predefined thresholds to determine state transitions. The states are defined strictly by algorithmic logic. State zero represents optimal operation where both accuracy and latency remain firmly within baseline parameters. State one represents minor degradation, flagging a drop in confidence while remaining operational. State two designates critical degradation where the system requires maximum allocation of processing cycles to resolve features, nearing a compute hang. The final two states are terminal. State q_f is triggered when either accuracy crashes below acceptable limits or latency spikes beyond maximum execution time. State q_{fatal} triggers when both metrics fail simultaneously, indicating a total hardware and software lockup.

5.3 Time-to-Failure Formulation

The primary quantitative output of the automaton is the Time-to-Failure metric. Unlike traditional reliability engineering models that measure time in continuous seconds or hours, this study discretizes time by the number of inference frames processed. The Time-to-Failure is explicitly defined as the consecutive sum of degraded CIFAR-10-C frames the system successfully navigates without triggering an absorbing final state. By quantifying resilience through exact frame counts, the methodology removes subjective interpretations of degradation curves. A model that achieves a higher frame count before crashing objectively demonstrates superior resource allocation and robustness against synthetic distribution shifts.

5.4 Bivariate RDI Computation

The index calculates transitions by mapping the severity levels built into the CIFAR-10-C dataset directly to dynamic evaluation thresholds. Rather than inventing a custom mathematical limit per image, which introduces unverified computer vision variables into a systems engineering study, the methodology relies on academically established benchmarks. The baseline averages established during the clean data phase serve as the anchor point. As the benchmark introduces level one through level five corruptions, the logic maps these predefined severities against the live inference data. If the current latency exceeds the predefined limit mapped to that severity tier, the state shifts downward. This provides a transparent, highly defensible transition logic loop.

5.5 Experimental Pipeline and Hardware Constraints

To rigorously test model stability in edge computing contexts, all training and inference benchmarking are strictly constrained to local, CPU-bound processing. The

physical hardware is locked to an Intel Core i5-12400 processor operating with 16 gigabytes of random access memory. The system explicitly excludes the use of any discrete graphics processing units or tensor accelerators. This hardware limitation guarantees that the computational efficiency dimension accurately reflects the physical bottlenecks inherent to local edge deployments and lightweight server nodes commonly utilized by regional businesses. Three architectures are passed through this pipeline: a standard ResNet-18 model to serve as the heavyweight baseline, compared directly against MobileNetV3 and ShuffleNet V2 to measure the exact Time-to-Failure variances of data-efficient structures.

5.6 Distributed Telemetry Architecture

To prevent input and output overhead from artificially inflating the inference latency measurements, the system relies on a distributed telemetry pipeline. If the local processor writes transition logs to its own hard drive while simultaneously executing neural network inference, the resulting latency spikes would invalidate the data. Therefore, local execution states and Time-to-Failure metrics are transmitted via a Tailscale secure encrypted network. These metrics are offloaded instantaneously to a centralized database hosted on a remote Hostinger subscription. This setup isolates the Intel i5 processor strictly for model inference and algorithmic transition calculations, ensuring high data integrity while proving the viability of distributed monitoring for decentralized networks.